游戏初始化和2、4的随机生成：
对于4*4的棋盘，我采用整型二维数组并且用0对其进行初始化。在游戏初始化时，我用 srand(time(0));
int res = rand() % 4;生成一对 0~3 的随机数作为随机出现数的坐标，加上 while()循环直到得到要求的坐标。
在之后的移动过程中随机出现的 2 或者 4 时，用双层 for 循环先找出数组中存在的 0 的个数 num，然后用 int res = rand() % num 生成一个 0~num-1 的随机数，定义一个整形 numpos=-1，再用一次双层 for 循环逐个找出数组中存在的 0，每遇到一个 0 就将 numpos++，看是否与所生成的随机数相等，如果相等就在这个位置赋值 2 或者 4 并退出循环，如果不相等就继续循环。要求 2 出现的该率大于 4，我采用 if (1 == rand()%3)，其中 rand()函数是上述随机生成 0~3 随机数的函数，如果随机数为 1 就出现 4，如果不是就出现 2，2 出现的概率是 75%，4 出现的概率是 25%。

每次操作后的清屏及输出：
清屏：system("cls");
输出：包装函数 show(arr，4);函数内用双层 for 循环实现数组的输出

上下左右加减操作：
用 while 循环实现不断地输入操作，每次输入后进入核心函数 core(c，arr，sp);，判断 c 的取值选择需要采用的操作，同时进行分数的计算。core(c，arr，sp)内部采用 switch 分支判断进入哪一种核心操作分支。核心操作的实现放在 Operation 文件中。每种操作的具体实现如下：选定好移动方向，第一步：逐行或逐列将所有的数先全部向该方向移动到不能移动为止（即到数组的边界或前一个位置已经有非 0 的数），并空位补 0。第二步：再逐行或逐列从选定方向开始将相邻的相同的数加起来，例如选上(W)，则逐列从第 0 行开始逐行往下判断，如果第 i 行和第 i+1 行的数相同，就将第 i 行的数重置为这两个数的和，并将第 i 行以下的数字全部上移同时在最后一个位置补上一个 0。

结束操作：
用双层 for 循环判断是否结束。若存在一个位置上的数为 2048，则直接跳出函数并且输出"恭喜您成功了！"；任何一个位置上都不为 0，则直接跳出函数并且输出"很遗憾失败了！"

趣味模式：
在经典模式下加入数字消除功能，在经典模式的代码基础上增加一个操作"Q"和一个"E"操作，用 if-else 结构将 Q 与 WASD 进行区分，如果键入 Q 或者 E 则进入趣味模式。数字消除则是采用双层 for 循环将数组中要清除的数字全部赋值为 0 即可实现，包装在 cleaning 文件中。数字改变同样用 for 循环逐个遍历数组中的所有元素，满足条件的元素进行重新赋值。

主文件：
用 switch 函数进行不同模式的选择，将各个模块打包应用于不同的 case 中即可实现完整功能。